

TÍNH TOÁN PHÂN TÁN

Bài toán lập lịch và các thuật toán cơ bản

Mô hình toán học, ví dụ tính toán
và liên hệ với Spark, MapReduce, Kafka

Bài giảng nhập môn / 50 trang

Nội dung và mục tiêu học tập

Nội dung

Sau bài học, có thể

Tác vụ độc lập	Chạy List Scheduling, LPT và tính makespan.
Phụ thuộc DAG	Tìm tác vụ sẵn sàng, đường găng và lập lịch hợp lệ.
Thời gian hoàn thành	So sánh FIFO, SPT, Smith và SRPT.
Tài nguyên dùng chung	Tính phần cấp công bằng và dominant share.
Partition Kafka	So sánh Round Robin, Range và tái cân bằng.

Mỗi thuật toán đi kèm một ví dụ có số liệu và điều kiện áp dụng.

Lập lịch nằm ở đâu trong hệ thống?

Thành phần	Quyết định	Mô hình liên quan
Spark DAGScheduler	Stage nào đã đủ điều kiện chạy?	Phụ thuộc DAG.
Spark TaskScheduler	Gửi task đến tài nguyên thực thi nào?	Phân công tác vụ, vị trí dữ liệu.
Hadoop YARN	Cấp tài nguyên cho ứng dụng/hàng đợi nào?	Phân bổ công bằng, giới hạn dung lượng.
Kafka consumer group	Consumer nào phụ trách partition nào?	Cân bằng tải, chi phí tái phân công.

Các tầng có thể cùng hoạt động, với những mục tiêu khác nhau.

Nguồn: tài liệu Spark [5], Hadoop [6] và Kafka [8] ở cuối bài.

Công việc, tác vụ và đơn vị thực thi

- **Job:** một công việc của người dùng, có thể gồm nhiều tác vụ.
- **Task:** đơn vị công việc được giao cho một tài nguyên thực thi.
- **Máy trong mô hình:** một đơn vị chỉ chạy một task tại một thời điểm.
- Một máy vật lý hoặc một Spark executor có thể có nhiều đơn vị thực thi song song.

Ví dụ: 3 máy vật lý, mỗi máy cho phép chạy 4 task đồng thời.

Mô hình đơn giản có $m = 3 \times 4 = 12$ đơn vị thực thi.

Phải xác định rõ “máy” trong mô hình trước khi áp dụng thuật toán.

Liên hệ kiến trúc: Spark cluster overview và TaskScheduler [5].

Ký hiệu và các mục tiêu thường gặp

n, m	Số tác vụ, số máy.
p_j	Thời gian xử lý tác vụ j .
r_j	Thời điểm tác vụ j đến.
S_j, C_j	Thời điểm bắt đầu, kết thúc.
w_j	Trọng số ưu tiên, $w_j > 0$.

Hoàn thành toàn bộ sớm nhất

$$\min C_{\max}, \quad C_{\max} = \max_j C_j.$$

Hoàn thành các công việc sớm

$$\min \sum_j C_j \quad \text{hoặc} \quad \min \sum_j w_j C_j.$$

Giảm tổng thời gian trong hệ thống

$$\min \sum_j (C_j - r_j).$$

Công bằng cần một định nghĩa riêng về phần tài nguyên được chia.

Những giả thiết quyết định bài toán

Câu hỏi

Các lựa chọn

Thời điểm biết tin	Offline: biết trước. Online: chỉ biết khi công việc đến.
Ngắt tác vụ	Không ngắt; hoặc ngắt rồi tiếp tục phần còn lại.
Năng lực máy	Cùng tốc độ; hoặc thời gian xử lý phụ thuộc máy.
Phụ thuộc	Độc lập; hoặc có thứ tự trước-sau theo DAG.
Chi phí phụ	Truyền dữ liệu, khởi động, chuyển trạng thái, lỗi máy.

Các ví dụ là dữ liệu giả định. Bảo đảm lý thuyết chỉ đúng dưới giả thiết đã nêu.

Bài toán phân công tác vụ độc lập

Có n tác vụ, m máy giống nhau. Tất cả tác vụ sẵn sàng tại $t = 0$. Mỗi tác vụ chạy liên tục trên đúng một máy. Bỏ qua chi phí truyền dữ liệu.

$$\min_{\{\mathcal{J}_1, \dots, \mathcal{J}_m\}} \max_{1 \leq i \leq m} \sum_{j \in \mathcal{J}_i} p_j,$$

trong đó $\mathcal{J}_1, \dots, \mathcal{J}_m$ là một phân hoạch của tập tác vụ.

Tác vụ	A	B	C	D	E	F	$m = 3.$
p_j	2	3	4	6	7	8	

Liên hệ: Các task độc lập trong một stage của Spark hoặc một pha Map.

Hai cận dưới cho thời gian tối ưu

Gọi OPT là makespan tối ưu, $W = \sum_j p_j$ là tổng khối lượng công việc. Khi đó

$$\text{OPT} \geq \max \left\{ \frac{W}{m}, \max_j p_j \right\}.$$

- W/m : các máy phải xử lý hết tổng công việc.
- $\max_j p_j$: một tác vụ không thể chạy nhanh hơn thời gian của chính nó.

Với ví dụ: $W = 30$, $m = 3$, $\max_j p_j = 8$, nên $\text{OPT} \geq 10$.

$$\underbrace{8+2}_{M_1} = 10, \quad \underbrace{7+3}_{M_2} = 10, \quad \underbrace{6+4}_{M_3} = 10.$$

Có lịch đạt cận dưới 10, do đó $\text{OPT} = 10$.

Thuật toán List Scheduling

Ý tưởng: giao tác vụ tiếp theo cho máy đang có tổng tải nhỏ nhất.

1. Khởi tạo tải $L_i = 0$ cho mỗi máy $i = 1, \dots, m$.
2. Duyệt các tác vụ theo thứ tự đã cho.
3. Với tác vụ j , chọn i có L_i nhỏ nhất.
4. Đặt $S_j = L_i$, $C_j = L_i + p_j$, rồi cập nhật $L_i \leftarrow L_i + p_j$.

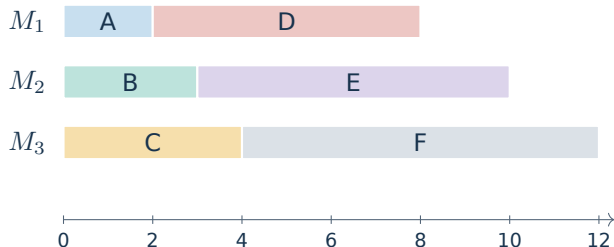
Nếu hòa, chọn máy có chỉ số nhỏ hơn để kết quả xác định.

Duyệt mọi máy: $O(nm)$. Dùng min-heap: $O(m + n \log m)$.

List Scheduling và bảo đảm xấp xỉ: Graham [1].

Ví dụ List Scheduling: thứ tự A, B, C, D, E, F

Sau tác vụ	L_1	L_2	L_3
A(2)	2	0	0
B(3)	2	3	0
C(4)	2	3	4
D(6)	8	3	4
E(7)	8	10	4
F(8)	8	10	12



$$C_{\max} = 12, \quad \frac{C_{\max}}{\text{OPT}} = 1,2.$$

Thứ tự xét tác vụ ảnh hưởng đến kết quả của thuật toán.

Vì sao List Scheduling có bảo đảm gần 2?

Gọi j là tác vụ kết thúc cuối cùng. Khi giao j , tải nhỏ nhất không vượt tải trung bình của các tác vụ đã giao, nên

$$S_j \leq \frac{W - p_j}{m}.$$

Suy ra

$$\begin{aligned} C_{\max} = S_j + p_j &\leq \frac{W}{m} + \left(1 - \frac{1}{m}\right) p_j \\ &\leq \text{OPT} + \left(1 - \frac{1}{m}\right) \text{OPT} = \left(2 - \frac{1}{m}\right) \text{OPT}. \end{aligned}$$

Thuật toán đơn giản, nhưng có bảo đảm cho mọi bộ dữ liệu hợp lệ.

Định lý List Scheduling: Graham [1]. Chứng minh dùng hai cận dưới ở trang 8.

Thuật toán LPT: xét tác vụ dài trước

Longest Processing Time first (LPT)

1. Sắp các tác vụ theo thời gian xử lý giảm dần.
2. Áp dụng List Scheduling theo thứ tự vừa sắp.

Trong ví dụ, thứ tự mới là

F(8), E(7), D(6), C(4), B(3), A(2).

Trực giác: bố trí các khối lớn trước, dùng khối nhỏ để lấp phần tải còn thiếu.

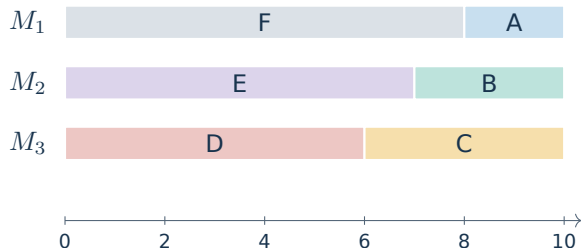
$$C_{\max}^{\text{LPT}} \leq \left(\frac{4}{3} - \frac{1}{3m} \right) \text{OPT}.$$

Chi phí sắp xếp: $O(n \log n)$. LPT cần biết hoặc ước lượng p_j .

Định lý LPT: Graham [1]. Bảo đảm dùng thời gian xử lý chính xác.

Ví dụ LPT: lịch đạt makespan 10

Sau tác vụ	L_1	L_2	L_3
F(8)	8	0	0
E(7)	8	7	0
D(6)	8	7	6
C(4)	8	7	10
B(3)	8	10	10
A(2)	10	10	10



$$C_{\max}^{\text{LPT}} = 10 = \text{OPT}.$$

Trong ví dụ này, LPT cải thiện makespan từ 12 xuống 10.

LPT có thể chưa tối ưu

Hai máy và năm tác vụ có thời gian 3, 3, 2, 2, 2.

LPT

Lịch tối ưu

$$M_1 : 3 + 2 + 2 = 7, \quad M_2 : 3 + 2 = 5.$$

$$M_1 : 3 + 3 = 6, \quad M_2 : 2 + 2 + 2 = 6.$$



$$\frac{C_{\max}^{\text{LPT}}}{\text{OPT}} = \frac{7}{6} = \frac{4}{3} - \frac{1}{3 \cdot 2}.$$

Liên hệ với task trong Spark và MapReduce

- Trong một stage đã sẵn sàng, nhiều task xử lý các partition khác nhau.
- Kích thước dữ liệu lệch nhau có thể làm thời gian chạy các task lệch nhau.
- List Scheduling và LPT là các mô hình cơ sở để phân tích, mô phỏng và so sánh cách phân công.

Khi dùng mô hình:

- Một đơn vị thực thi tương ứng một chỗ chạy task đồng thời.
- Cần xem thêm vị trí dữ liệu, bộ nhớ và chi phí shuffle.
- LPT không phải mô tả mặc định của Spark TaskScheduler.

Nguồn kiến trúc và nhiệm vụ TaskScheduler: [5]. Mô hình MapReduce: [7].

Bài toán lập lịch có quan hệ trước-sau

DAG là đồ thị có hướng không chu trình. Cho DAG $G = (V, E)$, trong đó đỉnh j là tác vụ có thời gian p_j .

$$(u, v) \in E \implies S_v \geq C_u.$$

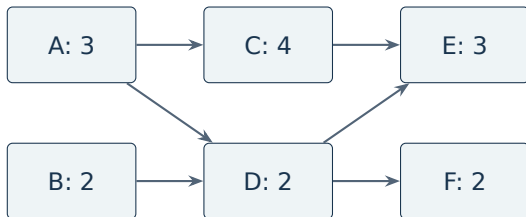
- Có m máy giống nhau, mỗi máy chạy một tác vụ tại một thời điểm.
- Tác vụ không bị ngắt. Bỏ qua chi phí truyền dữ liệu.
- Mục tiêu: tối thiểu hóa $C_{\max} = \max_j C_j$.

Tác vụ sẵn sàng: chưa chạy và mọi tác vụ tiền nhiệm đã hoàn thành.

Một lịch cần xác định cả máy lẫn thời điểm chạy.

Liên hệ: DAG của các stage trong Spark và phụ thuộc giữa các pha xử lý.

Ví dụ DAG với sáu tác vụ



Hai máy: $m = 2$.

$$W = 3 + 2 + 4 + 2 + 3 + 2 = 16.$$

Đường đi dài nhất:

$$A \rightarrow C \rightarrow E.$$

$$\text{Tổng độ dài: } 3 + 4 + 3 = 10.$$

$$\text{OPT} \geq \max\{W/m, \text{độ dài đường găng}\} = \max\{8, 10\} = 10.$$

List Scheduling cho DAG

1. Đếm số tiền nhiệm chưa hoàn thành của từng tác vụ.
2. Đưa các tác vụ có số đếm bằng 0 vào tập sẵn sàng.
3. Khi có máy rảnh, chọn một tác vụ sẵn sàng và cho chạy.
4. Khi tác vụ kết thúc, giảm số đếm của các tác vụ kế tiếp.
5. Bổ sung tác vụ mới sẵn sàng và lặp lại đến khi hoàn thành.

Quy tắc trong ví dụ: ưu tiên tên theo thứ tự A, B, C, D, E, F.

Khi nhiều tác vụ kết thúc cùng lúc, cập nhật tất cả rồi mới giao việc mới.

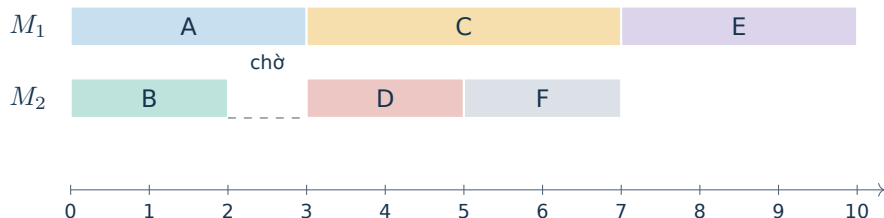
Không để máy rảnh nếu vẫn có tác vụ sẵn sàng có thể chạy.

Ví dụ DAG: các sự kiện trong quá trình chạy

Thời điểm	Vừa hoàn thành	Mới sẵn sàng	Quyết định
0	Chưa có	A, B	M_1 chạy A, M_2 chạy B.
2	B	Không có	M_2 chờ vì D còn phụ thuộc A.
3	A	C, D	M_1 chạy C, M_2 chạy D.
5	D	F	M_2 chạy F. E còn chờ C.
7	C, F	E	M_1 chạy E.
10	E	Không còn	Kết thúc.

Máy rảnh ở thời điểm 2 là do phụ thuộc dữ liệu, không phải thiếu tác vụ tổng thể.

Ví dụ DAG: biểu đồ Gantt và kết quả



$$C_{\max} = 10, \quad \text{OPT} \geq 10 \implies C_{\max} = \text{OPT}.$$

- E bắt đầu tại $\max\{C_C, C_D\} = \max\{7, 5\} = 7$.
- Hai máy chỉ bận đồng thời trong một phần của thời gian chạy.

Ưu tiên theo phần đường găng còn lại

Đặt $b(v)$ là độ dài đường đi dài nhất bắt đầu tại v , tính cả p_v .

$$b(v) = p_v + \max_{(v,u) \in E} b(u),$$

với giá trị lớn nhất bằng 0 khi v không có đỉnh kế tiếp.

1. Tính $b(v)$ theo thứ tự tô pô đảo, trong $O(|V| + |E|)$.
2. Khi chọn trong tập sẵn sàng, ưu tiên tác vụ có $b(v)$ lớn hơn.

v	A	B	C	D	E	F
$b(v)$	10	7	7	5	3	2

Ở $t = 3$, ưu tiên C trước D. Trong ví dụ hai máy, cả hai vẫn chạy ngay.

Đây là quy tắc chọn trong List Scheduling, không bảo đảm tối ưu mọi DAG.

Bảo đảm cho List Scheduling trên DAG

Gọi $W = \sum_j p_j$ và L là độ dài đường găng. Khi chi phí truyền dữ liệu bằng 0,

$$C_{\max} \leq \frac{W}{m} + \left(1 - \frac{1}{m}\right) L \leq \left(2 - \frac{1}{m}\right) \text{OPT}.$$

Ý tưởng chứng minh:

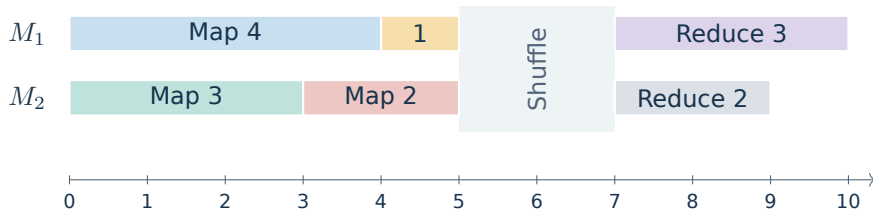
- Lần ngược từ tác vụ kết thúc cuối, mỗi lần chọn tiền nhiệm kết thúc muộn nhất.
- Ngoài thời gian chạy các tác vụ trên chuỗi này, tất cả máy đều bận.
- Tổng thời gian máy rảnh, cộng trên mọi máy, không quá $(m - 1)L$.

$$mC_{\max} = W + \text{tổng thời gian máy rảnh} \leq W + (m - 1)L.$$

Kết quả cổ điển của Graham [1]. Áp dụng cho máy giống nhau và tác vụ không bị ngắt.

Ví dụ MapReduce với rào chắn giữa các pha

Mô hình đơn giản: 2 chỗ chạy task, 4 task Map dài 4, 3, 2, 1. Sau toàn bộ Map là một pha shuffle dài 2, rồi 2 task Reduce dài 3, 2.



$$C_{\max} = \underbrace{5}_{\text{Map bằng LPT}} + \underbrace{2}_{\text{shuffle}} + \underbrace{3}_{\text{Reduce}} = 10.$$

Trong Hadoop thực tế, một phần shuffle có thể chồng lấp với Map. Ví dụ này cố ý dùng rào chắn để tính bằng tay.

Mô hình giảng dạy. Các pha Map, shuffle, sort, Reduce: Hadoop MapReduce Tutorial [7].

Vị trí dữ liệu: máy rảnh chưa chắc hoàn thành sớm

Một task cần 4 đơn vị xử lý. Dữ liệu nằm ở M_1 .

Lựa chọn	Chờ máy	Lấy dữ liệu	Kết thúc
Chạy tại M_1	3	0	$3 + 0 + 4 = 7$
Chạy tại M_2	0	5	$0 + 5 + 4 = 9$

Quy tắc tham lam: chọn máy có thời điểm hoàn thành dự kiến nhỏ nhất.

$$\hat{C}_i = \text{thời gian chờ}_i + \text{thời gian lấy dữ liệu}_i + p_j.$$

Nếu thời gian lấy dữ liệu ở M_2 chỉ là 1, nên chọn M_2 vì $1 + 4 = 5$.

Liên hệ: Ưu tiên dữ liệu cục bộ trong Spark/Hadoop. Ví dụ giả sử chờ, truyền và xử lý nối tiếp.

Vị trí ưu tiên của task: Spark DAGScheduler [5]. Quy tắc trên là mô hình dự báo đơn giản.

Bài toán trên một máy: giảm thời gian chờ

Một máy, các tác vụ độc lập đều sẵn sàng tại $t = 0$, không bị ngắt.

$$\min \sum_{j=1}^n C_j.$$

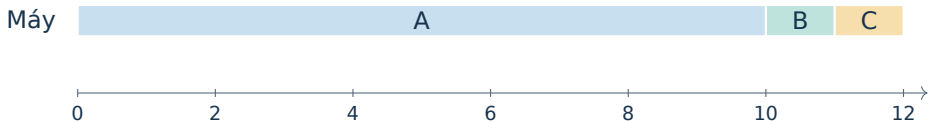
Tối thiểu hóa tổng này cũng tối thiểu hóa trung bình $\frac{1}{n} \sum_j C_j$.

Tác vụ	A	B	C
Thời gian xử lý p_j	10	1	1

- Tổng thời gian xử lý luôn là 12.
- Với lịch không có khoảng rảnh, makespan luôn là 12.
- Thứ tự chạy vẫn làm thay đổi mạnh tổng thời gian hoàn thành.

FIFO: xử lý theo thứ tự vào hàng đợi

Thuật toán: khi máy rảnh, lấy công việc ở đầu hàng đợi và chạy đến hết. Trong ví dụ, thứ tự hàng đợi là A, B, C.



$$(C_A, C_B, C_C) = (10, 11, 12), \quad \sum_j C_j = 33.$$

Trung bình thời gian hoàn thành: $33/3 = 11$.

Liên hệ: Spark có chế độ FIFO giữa các job. Cơ chế nhiều task của Spark phức tạp hơn hàng đợi một máy này.

Chính sách FIFO giữa các job trong một SparkContext: Spark Job Scheduling [5].

SPT: công việc ngắn chạy trước

Shortest Processing Time first (SPT)

1. Sắp các tác vụ theo p_j tăng dần.
2. Chạy lần lượt theo thứ tự đó. Chi phí sắp xếp: $O(n \log n)$.



$$(C_B, C_C, C_A) = (1, 2, 12), \quad \sum_j C_j = 15.$$

Trung bình giảm từ 11 xuống 5. Makespan vẫn bằng 12.

Chứng minh SPT tối ưu bằng đổi chỗ

Xét hai tác vụ kề nhau a, b bắt đầu từ thời điểm t , với $p_a > p_b$.

$$\text{Chạy } a \text{ rồi } b : C_a + C_b = 2t + 2p_a + p_b,$$

$$\text{Chạy } b \text{ rồi } a : C_b + C_a = 2t + 2p_b + p_a.$$

Đổi chỗ làm tổng giảm một lượng

$$p_a - p_b > 0.$$

- Các tác vụ trước cặp này không thay đổi.
- Tổng độ dài cặp vẫn bằng $p_a + p_b$, nên tác vụ phía sau không bị ảnh hưởng.

Loại bỏ mọi cặp ngược thứ tự sẽ thu được lịch SPT tối ưu.

SPT là trường hợp mọi trọng số bằng nhau của quy tắc Smith [2].

Smith: ưu tiên theo thời gian chia trọng số

Mỗi tác vụ có trọng số $w_j > 0$. Trên một máy, tất cả sẵn sàng tại $t = 0$:

$$\min \sum_j w_j C_j.$$

Thuật toán Smith: sắp theo p_j/w_j tăng dần rồi chạy lần lượt. Với hai tác vụ a, b , hiệu mục tiêu giữa thứ tự ab và ba là

$$w_b p_a - w_a p_b.$$

Do đó, cho a trước b không kém hơn nếu

$$\frac{p_a}{w_a} \leq \frac{p_b}{w_b}.$$

Cần cân đối thời gian xử lý với mức ưu tiên, không chỉ nhìn vào trọng số.

Quy tắc tối ưu cho một máy, không có thời điểm đến khác nhau: Smith [2].

Thời gian hoàn thành: FIFO, SPT, Smith và SRPT

Ví dụ Smith: tổng có trọng số giảm từ 55 xuống 30

Task	p_j	w_j	p_j/w_j
A	6	1	6
B	3	3	1
C	2	2	1

Theo Smith: B, C, A.

B và C có thể đổi chỗ vì cùng tỷ số.

Thứ tự A, B, C

$$(C_A, C_B, C_C) = (6, 9, 11),$$

$$\sum_j w_j C_j = 1 \cdot 6 + 3 \cdot 9 + 2 \cdot 11 = 55.$$

Thứ tự B, C, A

$$(C_B, C_C, C_A) = (3, 5, 11),$$

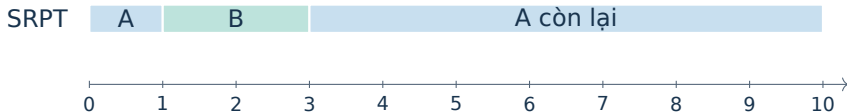
$$\sum_j w_j C_j = 3 \cdot 3 + 2 \cdot 5 + 1 \cdot 11 = 30.$$

Liên hệ: Mô hình ưu tiên công việc theo mức quan trọng. Trọng số này khác trọng số chia tài nguyên trong FAIR.

SRPT: công việc đến theo thời gian

Một máy, biết p_j khi công việc đến. Cho phép ngắt và tiếp tục, không mất chi phí. **SRPT:** luôn chạy công việc đã đến có thời gian xử lý còn lại nhỏ nhất.

$$A : (r_A, p_A) = (0, 8), \quad B : (r_B, p_B) = (1, 2).$$



$$\sum_j (C_j - r_j) = (10 - 0) + (3 - 1) = 12.$$

FIFO không ngắt cho A chạy $[0, 8]$, B chạy $[8, 10]$, nên tổng là $8 + 9 = 17$.

SRPT tối ưu tổng thời gian trong hệ thống dưới các giả thiết trên.

SRPT: Schrage [3]. Kết quả không áp dụng trực tiếp khi có trọng số hoặc chi phí ngắt.

Liên hệ với hàng đợi công việc trong hệ thống

Chính sách	Thông tin cần có	Cách dùng mô hình
FIFO	Thứ tự đến.	Chính sách cơ sở để triển khai.
SPT / Smith	Ước lượng thời gian, thêm trọng số với Smith.	So sánh độ trễ của các yêu cầu.
SRPT	Thời gian còn lại và khả năng ngắt.	Mô hình lý tưởng cho công việc đến động.
FAIR	Phần tài nguyên của job/pool.	Chia sẻ tài nguyên giữa nhiều công việc.

Spark có FIFO và FAIR. Không thể suy ra rằng bất FAIR sẽ thực thi SPT hoặc SRPT.

Spark Job Scheduling [5]. Các ví dụ một máy ở trang 25–31 là mô hình phân tích.

Bài toán chia một loại tài nguyên

Có tổng tài nguyên B . Người dùng i có nhu cầu tối đa d_i . Chọn lượng cấp x_i sao cho

$$0 \leq x_i \leq d_i, \quad \sum_i x_i \leq B.$$

Công bằng max-min: không thể tăng phần cấp của một người mà không giảm phần cấp của một người đang nhận ít hơn hoặc bằng người đó. Ví dụ: $B = 12$ đơn vị CPU và ba người dùng có nhu cầu

$$(d_1, d_2, d_3) = (2, 8, 8).$$

Chia đều 4 cho mỗi người vượt nhu cầu của người thứ nhất. Cần phân phối lại phần dư.

Liên hệ: Phân bổ giữa các người dùng hoặc hàng đợi trong một cụm dùng chung.

Progressive filling: tăng đều phân cấp

1. Ban đầu, cấp 0 cho mọi người dùng.
2. Tăng đều phân cấp của những người chưa đạt nhu cầu.
3. Khi một người đạt nhu cầu, giữ nguyên phân cấp của người đó.
4. Tiếp tục đến khi hết tài nguyên hoặc mọi nhu cầu đều đủ.

Giai đoạn	x_1	x_2	x_3	Tổng
Tăng đều đến mức 2	2	2	2	6
Giữ người 1, chia 6 còn lại	2	5	5	12

$$x_i = \min\{d_i, \lambda\}, \quad \lambda = 5.$$

Với nhu cầu đã biết, có thể sắp các d_i để tính các mức tăng trong $O(n \log n)$.

Bài toán chia nhiều loại tài nguyên

Có d loại tài nguyên, loại r có dung lượng R_r . Một task của người dùng i cần a_{ir} đơn vị tài nguyên loại r .

$$\sum_i x_i a_{ir} \leq R_r \quad (r = 1, \dots, d).$$

x_i là số task của người dùng i được cấp tài nguyên đồng thời. **Dominant share** của người dùng i :

$$s_i = \max_r \frac{x_i a_{ir}}{R_r}.$$

DRF: thực hiện công bằng max-min trên các dominant share.

Liên hệ: Người dùng có thể thiên về CPU hoặc RAM. Chia một loại tài nguyên có thể làm loại khác quá tải.

Dominant Resource Fairness (DRF): Ghodsi và cộng sự [4].

Ví dụ DRF: hai người dùng có nhu cầu khác nhau

Cụm có **9 CPU** và **18 GB RAM**. Hai người dùng có đủ task để tiếp tục nhận tài nguyên.

Người dùng	CPU/task	RAM/task	Tài nguyên trội
A	1	4 GB	RAM: $4/18 > 1/9$.
B	3	1 GB	CPU: $3/9 > 1/18$.

$$s_A = \frac{2x_A}{9}, \quad s_B = \frac{x_B}{3}.$$

Ràng buộc khả thi:

$$x_A + 3x_B \leq 9, \quad 4x_A + x_B \leq 18.$$

Trong mô hình liên tục, x_A, x_B có thể là số thực không âm.

Ví dụ DRF: tăng đều dominant share

Đặt dominant share chung bằng s . Khi đó

$$x_A = \frac{9}{2}s, \quad x_B = 3s.$$

Thay vào hai ràng buộc:

$$\underbrace{\frac{9}{2}s + 9s}_{\text{CPU}} \leq 9 \implies s \leq \frac{2}{3}, \quad \underbrace{18s + 3s}_{\text{RAM}} \leq 18 \implies s \leq \frac{6}{7}.$$

CPU đầy trước, nên $s = 2/3$ và

$$x_A = 3, \quad x_B = 2.$$

Tổng sử dụng: $3 + 3 \cdot 2 = 9$ CPU và $4 \cdot 3 + 2 = 14$ GB RAM.

Hai người có cùng dominant share, nhưng số task được cấp khác nhau.

DRF theo từng task: một cách cấp phát đơn giản

Quy tắc: chọn người có s_i nhỏ nhất trong số có task vừa tài nguyên còn lại. Cấp thêm một task. Nếu hòa, chọn A trước.

Bước	Cấp cho	(x_A, x_B)	(s_A, s_B)	CPU dùng	RAM dùng
0	Chưa cấp	(0, 0)	(0, 0)	0	0
1	A	(1, 0)	(2/9, 0)	1	4
2	B	(1, 1)	(2/9, 1/3)	4	5
3	A	(2, 1)	(4/9, 1/3)	5	9
4	B	(2, 2)	(4/9, 2/3)	8	10
5	A	(3, 2)	(2/3, 2/3)	9	14

Dừng vì hết CPU, trùng nghiệm liên tục ở trang 37.

Task nguyên hoặc máy riêng biệt có thể làm thay đổi bảo đảm lý thuyết.

Ý tưởng chọn dominant share nhỏ nhất: [4]. Bảng trên là ví dụ cấp phát theo task nguyên.

Liên hệ với YARN và Spark FAIR

- **YARN Fair Scheduler** chia tài nguyên giữa các ứng dụng và hàng đợi. Có thể cấu hình DRF để xét CPU cùng bộ nhớ.
- **YARN Capacity Scheduler** quản lý phần dung lượng được bảo đảm và các giới hạn của hàng đợi.
- **Spark FAIR** chia sẻ tài nguyên giữa các job/pool bên trong một SparkContext.

Hai tầng khi Spark chạy trên YARN:

1. YARN cấp tài nguyên cho các ứng dụng Spark.
2. Mỗi ứng dụng Spark dùng tài nguyên đó để chạy các task của mình.

Nguồn: Spark Job Scheduling [5], YARN Fair Scheduler và Capacity Scheduler [6].

Bài toán phân công partition trong consumer group

Partition là một phần dữ liệu của topic. Consumer là tiến trình đọc/xử lý. Xét một **consumer group truyền thống**: 6 partition, 3 consumer. Mỗi partition giao cho đúng một consumer đủ điều kiện.

Partition	P_1	P_2	P_3	P_4	P_5	P_6
Tải giả định a_j	8	7	6	3	2	1

Đặt $\mathcal{P}_i$ là tập partition của consumer i và

$$L_i = \sum_{j \in \mathcal{P}_i} a_j.$$

- **Cân bằng số lượng:** các $|\mathcal{P}_i|$ gần nhau.
- **Cân bằng tải:** làm nhỏ $\max_i L_i$.

Phạm vi: consumer group dùng cơ chế gán partition, không phải share group. Các assignor: [8].

Round Robin: phân công lần lượt theo vòng

Giả sử mọi consumer đăng ký cùng các topic. Xét các partition theo thứ tự $P_1, \dots, P_6$. **Thuật toán:** lần lượt giao cho C_1, C_2, C_3 , rồi quay lại C_1 .

Consumer	Partition nhận	Số lượng	Tổng tải
C_1	P_1, P_4	2	$8 + 3 = 11$
C_2	P_2, P_5	2	$7 + 2 = 9$
C_3	P_3, P_6	2	$6 + 1 = 7$

$$\max_i L_i = 11, \quad \max_i |\mathcal{P}_i| - \min_i |\mathcal{P}_i| = 0.$$

Số partition bằng nhau, nhưng tải xử lý vẫn khác nhau.

RoundRobinAssignor của Kafka [8]. Ví dụ dùng cùng subscription để vòng phân công không phải bỏ qua consumer.

Range: chia các đoạn liên tiếp theo từng topic

Một topic có n partition, m consumer cùng đăng ký topic đó.

1. Sắp partition và consumer theo thứ tự xác định.
2. Đặt $q = \lfloor n/m \rfloor$, $r = n \bmod m$.
3. r consumer đầu nhận $q + 1$ partition, các consumer còn lại nhận q .

Consumer	Đoạn nhận trong ví dụ	Tổng tải
C_1	P_1, P_2	$8 + 7 = 15$
C_2	P_3, P_4	$6 + 3 = 9$
C_3	P_5, P_6	$2 + 1 = 3$

Range cân bằng số lượng theo từng topic. Nhiều topic có thể tạo lệch tổng số partition.

RangeAssignor của Kafka [8]. Ví dụ bỏ qua các tùy chọn vị trí vật lý/rack.

LPT theo tải: một phương án để so sánh

Giả sử biết tải a_j và mọi consumer có cùng năng lực, cùng quyền nhận partition.

1. Sắp partition theo tải giảm dần.
2. Giao partition tiếp theo cho consumer đang có tải nhỏ nhất.

Consumer	Partition nhận	Tổng tải
C_1	P_1, P_6	$8 + 1 = 9$
C_2	P_2, P_5	$7 + 2 = 9$
C_3	P_3, P_4	$6 + 3 = 9$

Tổng tải là 27, nên cần dưới bằng $27/3 = 9$. Phân công trên tối ưu cho ví dụ.

Phương án LPT cần đo tải và xây dựng cơ chế phân công tùy biến.

Tái cân bằng khi một consumer rời nhóm

Bắt đầu từ phân công Round Robin ở trang 41. Consumer C_2 rời nhóm. Nếu phân công lại toàn bộ theo vòng C_1, C_3 , ta được:

Partition	Consumer cũ	Consumer mới	Có chuyển?
P_1	C_1	C_1	Không
P_2	C_2	C_3	Có
P_3	C_3	C_1	Có
P_4	C_1	C_3	Có
P_5	C_2	C_1	Có
P_6	C_3	C_3	Không

Có 4 partition đổi consumer. Trong đó chỉ P_2, P_5 bắt buộc chuyển vì mất consumer cũ.

Phân công lại có thể làm mất cache hoặc phải chuyển/khôi phục trạng thái.

Động cơ giữ phân công cũ khi rebalance: StickyAssignor [8].

Nguyên tắc sticky: giữ lại phân công còn hợp lệ

Thuật toán minh họa cho ví dụ:

1. Giữ $C_1 : \{P_1, P_4\}$ và $C_3 : \{P_3, P_6\}$.
2. Xét partition mất chủ theo thứ tự P_2, P_5 .
3. Giao cho consumer có ít partition hơn. Nếu hòa, chọn C_1 .

Consumer	Sau khi bổ sung	Số lượng	Tải
C_1	P_1, P_4, P_2	3	18
C_3	P_3, P_6, P_5	3	9

Chỉ chuyển 2 partition. Đây là tối thiểu vì P_2, P_5 bắt buộc đổi chủ.

Ví dụ thể hiện nguyên tắc sticky. Cài đặt Kafka còn xử lý subscription và cân bằng tổng quát.

Kafka StickyAssignor ưu tiên cân bằng số partition rồi giữ phân công cũ [8].

Cân bằng tải và giảm di chuyển là hai tiêu chí

Vẫn giữ các partition cũ, nhưng giao P_5 cho C_1 , P_2 cho C_3 :

$$L_1 = 8 + 3 + 2 = 13, \quad L_3 = 6 + 1 + 7 = 14.$$

Phương án	Tải lớn nhất	Số partition chuyển D
Round Robin lại toàn bộ	16	4
Giữ phân công, hòa chọn C_1	18	2
Giữ phân công, xét thêm tải	14	2

Một mô hình mở rộng có thể chọn mục tiêu

$$\min_f \left[\max_i L_i(f) + \lambda D(f) \right], \quad \lambda \geq 0,$$

trong đó f là phân công mới. Hệ số λ đổi chi phí chuyển về cùng thang đo với tải.

Bảng chọn thuật toán theo bài toán

Bài toán	Thuật toán cơ bản	Điều kiện cần nhớ
Tác vụ độc lập	List Scheduling / LPT	Giảm makespan, máy giống nhau.
Tác vụ có phụ thuộc	List Scheduling trên DAG	Chỉ chọn tác vụ đã sẵn sàng.
Một máy, giảm $\sum C_j$	SPT	Tất cả có sẵn từ đầu.
Một máy, giảm $\sum w_j C_j$	Smith	Biết p_j, w_j , có sẵn từ đầu.
Một máy, tác vụ đến động	SRPT	Biết độ dài, ngắt không mất phí.
Chia tài nguyên	Tăng đều / DRF	Định nghĩa rõ nhu cầu và công bằng.
Gán partition theo số lượng	Round Robin / Range	Phụ thuộc subscription.
Giảm di chuyển partition	Nguyên tắc sticky	Ưu tiên giữ phân công hợp lệ.

Bài tập ngắn và đáp số

Bài 1. Hai máy, các tác vụ có thời gian 5, 4, 3, 2, 2. Chạy LPT, hòa chọn máy 1. Tính makespan và tìm một lịch tốt hơn.

Đáp số: LPT cho tải (9, 7). Lịch (5 + 3, 4 + 2 + 2) có tải (8, 8) và tối ưu.

Bài 2. Có 15 đơn vị tài nguyên, nhu cầu của ba người là (3, 4, 20). Dùng progressive filling để tìm phân cấp công bằng.

Đáp số: Tăng đến (3, 3, 3), rồi (3, 4, 4), cuối cùng (3, 4, 8).

Bài 3. Consumer rời nhóm đang giữ 4 partition. Ít nhất bao nhiêu partition phải đổi consumer nếu vẫn giữ nguyên các partition?

Đáp số: Ít nhất 4. Có đạt được đồng thời với cân bằng hay không còn tùy subscription và ràng buộc.

Tài liệu tham khảo về thuật toán

[1] R. L. Graham (1969).

Bounds on Multiprocessing Timing Anomalies.

SIAM Journal on Applied Mathematics, 17(2), 416–429. [DOI](#)

[2] W. E. Smith (1956).

Various Optimizers for Single-Stage Production.

Naval Research Logistics Quarterly, 3(1-2), 59–66. [DOI](#)

[3] L. Schrage (1968).

A Proof of the Optimality of the Shortest Remaining Processing Time Discipline.

Operations Research, 16(3), 687–690. [DOI](#)

[4] A. Ghodsi và cộng sự (2011).

Dominant Resource Fairness: Fair Allocation of Multiple Resource Types.

NSDI 2011. [Bài báo trên USENIX](#)

Tài liệu chính thức của các hệ thống

[5] Apache Spark.

[Job Scheduling \(3.5.6\)](#); [Cluster Mode Overview](#); [TaskScheduler API](#); [DAGScheduler source](#).

[6] Apache Hadoop YARN (3.4.0).

[Fair Scheduler](#); [Capacity Scheduler](#).

[7] Apache Hadoop MapReduce (3.4.0).

[MapReduce Tutorial: task, shuffle, sort và Reduce](#).

[8] Apache Kafka, Consumer API (3.7.2).

[RoundRobinAssignor](#); [RangeAssignor](#); [StickyAssignor](#).

Phạm vi đối chiếu: các phiên bản nêu trên. Cấu hình và giao thức của phiên bản khác có thể thay đổi.